ViBridge-VQA: Bridging Global and Local Visual Tokens for Parameter-Efficient Vietnamese Visual Question Answering

Nguyen Trung Quoc, Nguyen Anh Tuong, Phan Ba Duc, Tran Dac Thinh, Dang Duy Lan, Nguyen Quoc Thinh, and Tung Le (✉)

Faculty of Information Technology, University of Science, Ho Chi Minh City, Vietnam
Vietnam National University, Ho Chi Minh City, Vietnam
{22120071,22120182,22120301,22120347,
22120348,22120412}@student.hcmus.edu.vn, lttung@fit.hcmus.edu.vn

Abstract. Vision–language models (VLMs) such as Vintern-1B bring strong multimodal abilities to Vietnamese. However, adapting them to a new Visual Question Answering (VQA) benchmark usually requires fine-tuning on many high-resolution image tiles or training a new architecture. We propose **ViBridge-VQA**, which improves Vintern-1B without updating any of its pre-trained weights. From a single image view, the frozen decoder receives two kinds of visual tokens. *Global* tokens are produced from the `[CLS]` vector by a small trainable bridge. *Local* tokens come from Vintern-1B's own pre-aligned projector and are pooled to a budget chosen as the knee of the accuracy–cost Pareto front. On the reasoning-oriented AutoViVQA benchmark, ViBridge-VQA trains 1.35% of the parameters and reaches 54.6 F1 on a stricter image-disjoint split, comparable to the fine-tuned Vintern-1B, with 36 times fewer visual tokens. Adding a lightweight decoder LoRA raises F1 to 58.2. Our analysis shows that reusing the pre-trained projector is more effective than learning a new connector. The local tokens also complement decoder adaptation, and their gain carries over to three other Vietnamese VQA datasets.

Keywords: Vietnamese Visual Question Answering · Vision–Language Models · Parameter-Efficient Learning · Visual Token Budget · Generative VQA

1 Introduction

Visual Question Answering (VQA) asks a model to answer a natural-language question about an image. The task has advanced rapidly with large vision–language models (VLMs) such as BLIP-2 [13], Flamingo [1] and LLaVA [20], which connect a pre-trained vision encoder to a pre-trained language model to generate free-form answers. Vietnamese has followed this trend with the

(✉)Corresponding author: Tung Le – lttung@fit.hcmus.edu.vn

open model Vintern-1B [8], which connects an InternViT-300M encoder [7] to a Qwen2.5-0.5B decoder [27]. Benchmarks such as ViVQA [30] and the reasoning-oriented AutoViVQA [22] measure progress in this low-resource setting.

However, building a strong model for a new Vietnamese benchmark is still expensive. The common recipe fine-tunes an existing VLM: the fine-tuned Vintern-1B baseline of AutoViVQA adds LoRA adapters to the language model and splits each image into up to six high-resolution tiles plus a thumbnail. On the AutoViVQA test images this gives seven views, so its decoder reads 1,792 visual tokens per image. Another option is a task-specific architecture, such as ViMoE-VQA [21] with its sparse mixture-of-experts fusion module. A cheaper option is to keep the whole VLM frozen and train only a small connector, which usually follows one of two designs. A *global-only bridge* maps the global image vector to a few tokens, which is cheap but loses local details such as small objects, printed text and spatial relations. A *learned patch compressor*, such as a Q-Former, keeps these details but must learn a new mapping from visual features to the decoder, which is difficult with limited Vietnamese data.

Vintern-1B already offers a third option: its projector was pre-trained to map image regions into the input space of the decoder, so this alignment does not need to be learned again. Meanwhile, the model discards the global `[CLS]` vector of its encoder, which summarises the whole scene. We therefore study two questions. With limited labelled data, is reusing the pre-trained alignment of a VLM more effective than learning a new connector? And how many global and local tokens does the frozen decoder need?

To this end, we propose **ViBridge-VQA**, which gives the frozen decoder of Vintern-1B two complementary kinds of visual tokens from a single image view. *Local* tokens reuse the frozen projector, average-pooled to a chosen budget and placed in the original image slot. The decoder thus receives region features in the representation it was trained to read. *Global* tokens, produced from the `[CLS]` vector by a small trainable bridge, add the scene-level information that Vintern-1B ignores. This bridge is the only trained part. Since every additional token increases inference cost, we treat the number of tokens as a budget and select it as the knee of the accuracy–cost Pareto front. ViBridge-VQA performs on par with the fine-tuned Vintern-1B without updating any of its weights, while its decoder reads far fewer visual tokens.

In summary, our main contributions are as follows:

- We propose **ViBridge-VQA**, a global–local bridge that combines trainable global tokens with local tokens from the frozen VLM's own pre-trained projector. It can also be used together with a lightweight decoder LoRA.
- We treat the number of visual tokens as an explicit **accuracy–cost budget**, selected on validation as the knee of the Pareto front.
- We **analyse where the gains come from**: local tokens matter more than global ones, reusing the projector beats learning a new connector, and local tokens complement decoder adaptation, and their gain carries over to other Vietnamese VQA datasets.

2 Related Work

Early VQA systems classified over a fixed answer set. Current generative VLMs instead produce free-form answers with an autoregressive language model [13,1,20]. Vietnamese VQA datasets include ViVQA [30], the synthetic ViCLEVR [32], the book-cover ViOCRVQA [26] and the scene-text ViTextVQA [24]. AutoViVQA [22] contributes 37,077 questions on 19,411 COCO images [18], generated by large language models, filtered by an ensemble of validators and annotated with reasoning categories. On the modelling side, BARTPhoBEiT [31] and AViVQA-TranConI [23] combine pre-trained visual and Vietnamese language representations. Vintern-1B [8] brings an instruction-tuned VLM to Vietnamese, and ViMoE-VQA [21] routes multimodal tokens to sparse experts for generative VQA on AutoViVQA. AutoViVQA and ViMoE-VQA are earlier work of our group. Here we use the AutoViVQA data but no component of ViMoE-VQA, and the method, the image-disjoint split and all results are new. These approaches either train a new model or fine-tune a VLM on multi-tile inputs, whereas we keep every pre-trained weight of Vintern-1B frozen and use a single image view.

Training only a connector between a frozen vision encoder and a frozen language model is a common way to build VLMs efficiently. Connectors differ mainly in what they must learn. Frozen [33] learns a visual prefix from a global image representation. The Q-Former of BLIP-2 [13] learns queries that compress the patch features, which preserves details but requires a large module (188M parameters) pre-trained on millions of image–text pairs. LLaVA [20,19] projects every patch token with a small MLP, which preserves locality but passes hundreds of tokens to the decoder. Honeybee [4] proposes locality-preserving abstractors, and InternVL [7], on which Vintern-1B is built, merges neighbouring patches before its MLP projector. In all of these models, the mapping from patches to the decoder is learned during large-scale pre-training. ViBridge-VQA does not learn this mapping again: it reuses the projector already pre-trained in the VLM and trains only a small global bridge (12.9M parameters) on about 26k task questions.

The number of visual tokens largely determines the cost of the decoder. Inference-optimal scaling of VLMs [14] shows that a few tokens can suffice for many tasks. Matryoshka multimodal models [3] work at several nested pooled budgets and show that the required budget depends on the task. Other work reduces the visual tokens directly. DeCo [36] replaces the Q-Former with adaptive average pooling of the patch tokens and reports better results. TokenPacker [15] learns a compact projector, while LLaVA-PruMerge [29] and FastV [5] prune or merge the visual tokens of an existing model, and VisionZip [35] keeps those that receive the most attention from the `[CLS]` token. Like DeCo, we use simple average pooling, but we apply it to the output of an already pre-trained projector and keep the whole VLM frozen. We also choose the budget with explicit criteria: the knee of the Pareto front [28] and TOPSIS [12]. Parameter-efficient methods such as adapters [10], prefix tuning [16] and LoRA [11] reduce the cost of adaptation from the other side, by training a small number of additional weights in the decoder. We use decoder LoRA both as a baseline and as a complement, and

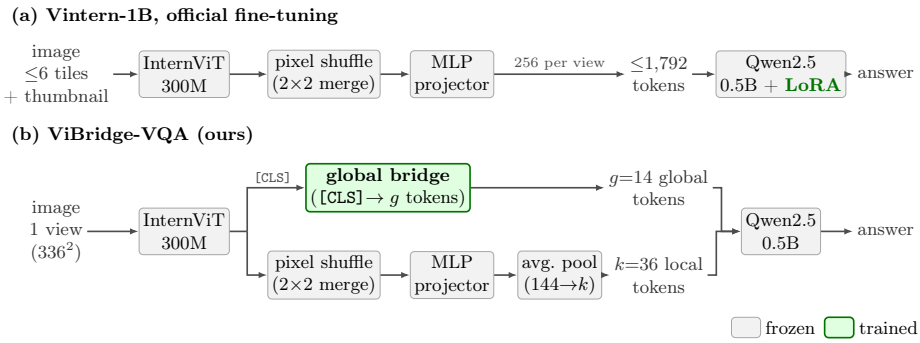


Fig. 1: Overview of (a) the official fine-tuning of Vintern-1B, which encodes up to seven views, and (b) ViBridge-VQA, which encodes one view. In both, the pixel shuffle merges each 2×2 block of patches and the pre-trained MLP projector maps it to the decoder space, which gives 144 region tokens for our 336×336 view. ViBridge-VQA average-pools them to k local tokens and adds g global tokens from the `[CLS]` vector. The selected budget, $g = 14$ and $k = 36$, gives 50 visual tokens.

ask whether a small training budget is better spent on the visual tokens or on adapting the decoder.

3 Proposed Method

We formulate VQA as conditional answer generation. Given an image I and a Vietnamese question q, the model generates a short free-form answer $y = (y_1, \ldots, y_T)$. ViBridge-VQA builds on Vintern-1B [8] and keeps all of its pre-trained weights frozen. Instead of adapting the model, it changes the visual input of its decoder.

3.1 Overall Architecture

As shown in Fig. 1, ViBridge-VQA keeps the three components of Vintern-1B: the InternViT-300M vision encoder (24 layers, width 1024), its two-layer MLP projector P, and the Qwen2.5-0.5B-Instruct decoder (24 layers, width 896). Each image is resized to one 336×336 view, i.e., 24×24 patches of 14×14 pixels. InternViT, pre-trained at 448×448, bicubically interpolates its position embeddings to this resolution, which roughly halves the encoder cost (191 instead of 362 GMACs per view). The encoder returns one vector per patch and a global `[CLS]` vector $v \in \mathbb{R}^{1024}$. Vintern-1B uses only the patch vectors: it merges every 2×2 block of neighbouring patches into one 4096-dimensional vector (pixel shuffle [6]). The projector P, pre-trained to align the two backbones, maps each merged vector into the decoder input space. At our resolution, this yields a 12×12 grid of 144 region tokens. ViBridge-VQA uses both outputs of the encoder.

3.2 Global Tokens

The `[CLS]` vector summarises the whole scene, but Vintern-1B never passes it to the decoder. A small trainable bridge f_ϕ expands it into g tokens with two linear heads:

$$f_\phi(v) = \big[\, W_a v + b_a;\ \mathrm{reshape}_{(g-1)\times 896}(W_e v + b_e) \,\big] \in \mathbb{R}^{g\times 896}, \tag{1}$$

where $W_a \in \mathbb{R}^{896\times 1024}$ and $W_e \in \mathbb{R}^{(g-1)896\times 1024}$. The first head produces an *anchor* token, and the second produces $g-1$ complementary tokens that present the global content from different learned directions. The bridge has $1025 \cdot 896 \cdot g$ parameters, i.e., 12,857,600 for $g = 14$, and its g tokens are placed before the prompt. To choose g, we varied the bridge alone from 4 to 20 tokens. F1 stops increasing at $g = 14$ (Section 4.5), which we use in ViBridge-VQA. We keep $g = 8$ for our Global-only model, as it matches the eight learned queries of the Tile-Attention and light Q-Former bridges it is compared with (Section 4.4). The local tokens are combined with both values, and the final pair (g, k) is selected with the criterion of Section 3.5.

3.3 Local Tokens

To give the decoder region-level detail, ViBridge-VQA also passes local tokens taken directly from the 144 region tokens of the frozen projector P. To control their number, we average-pool the 12×12 grid to an $s \times s$ grid, so that $k = s^2$ with $s \in \{1, 3, 6, 12\}$. Thus $k = 144$ keeps every region token, and $k = 36$ averages each 2×2 block. The pooled tokens replace the embeddings of k image-context placeholders inside Vintern-1B's own image slot, `<img>` $\cdots$ `</img>`, in the prompt. Because they come from the pre-trained projector rather than from a newly learned projection, the local tokens stay in the representation that the decoder was trained to read. Producing them requires no parameters.

3.4 Training Objective

Only the bridge parameters ϕ are trained. The vision encoder, the projector and the decoder remain frozen, and no adapter is added to the decoder. We minimise the cross-entropy of the answer tokens:

$$\mathcal{L} = -\sum_{t=1}^{T} \log p\big(y_t \mid y_{<t},\, f_\phi(v),\, \mathrm{pool}_s(P(x)),\, q\big), \tag{2}$$

where x denotes the merged patch vectors. ViBridge-VQA can also be combined with decoder adaptation. In that variant, a rank-16 LoRA [11] on the query, key, value and output projections of every decoder layer is trained jointly with the bridge and adds 2,162,688 parameters. Throughout the paper, percentages of trained parameters are relative to all parameters of the model, which comprise the frozen components of Vintern-1B (about 0.94B, including the vision encoder, projector, embeddings and output layer) plus the trained modules. With $g = 14$, the bridge alone trains 12.86M parameters (1.35%), and the bridge with LoRA 15.02M (1.57%).

3.5 Choosing the Token Budget

Since the cost of the decoder grows with the number of visual tokens $g + k$, we treat the pair (g, k) as a budget to be chosen. In our experiments, F1 increases with every added token among the configurations with local tokens. All of them thus lie on the accuracy–token Pareto front, and the front alone does not single one out. We therefore select its *knee* [28], the point after which additional tokens bring diminishing returns. Here the accuracy a is the validation F1 and the cost c is the number of visual tokens. We first normalise c and a to $[0, 1]$ over the front. The knee is the configuration that maximises $\hat{a} - \hat{c}$, i.e., the one furthest above the line joining the cheapest and the most accurate configuration. The knee is computed on the validation split only, and we check the choice with the multi-criteria method TOPSIS [12], also on validation. Test results are computed for every configuration only to be reported, and no choice is based on them.

4 Experiments

4.1 Dataset and Evaluation Metrics

Dataset. We use AutoViVQA [22], a Vietnamese VQA benchmark built on MS COCO images [18], in which each question has five short free-form answers and a reasoning type. We remove 273 duplicated (image, question) pairs and 97 questions without a reasoning label. The remaining 36,707 questions are split 70/15/15 by image (seed 42), so no test image is seen in training. This gives 25,776 / 5,463 / 5,468 questions on 13,579 / 2,909 / 2,910 images. The split is stricter than a question-level split: it lowers the F1 of Global-only by about 0.5 points.

Metrics. Following AutoViVQA and ViMoE-VQA, we report Accuracy (exact match), word-level Precision, Recall and F1, BLEU [25], ROUGE-L [17], METEOR [2] and CIDEr [34], all against the five reference answers. For each question, Precision, Recall and F1 compare the sets of words of the answer and of the reference that gives the highest F1, and are then averaged over questions. Accuracy counts the answers identical to one of the references, ignoring case, and BLEU is corpus-level BLEU-4 on whitespace tokens.

4.2 Experimental Setup

Implementation details. The vision encoder, projector and decoder come from Vintern-1B-v3.5 [8] and stay frozen. We train with AdamW (learning rate 2×10^{-4}, effective batch size 8) on the first reference answer. ViBridge-VQA is trained for two epochs, after which the validation loss and CIDEr stop improving, and the variants with a decoder LoRA for one epoch, in which the bridge and the LoRA are trained jointly from scratch. Global-only + LoRA, used in Fig. 2 and Table 5, is trained for three epochs, its best setting (+1.3 validation F1 over one epoch). Decoding is greedy, with at most 50 new tokens, and every model fits on

Table 1: Performance comparison on the AutoViVQA test set. Baseline results are as reported in AutoViVQA [22] and ViMoE-VQA [21] on the original AutoViVQA split; ours are the mean ± std over three seeds on our image-disjoint split (Section 4.1). Best results per column are in bold. All values ×100.

	Model	Accuracy	Precision	Recall	F1	BLEU	ROUGE-L	METEOR	CIDEr
Fine-tuned	ViT5_ViT	7.97	46.84	50.33	48.52	4.13	46.89	31.02	72.68
	BARTPhoBEiT	8.81	45.30	46.48	45.88	4.33	44.83	24.57	**188.96**
	Vintern-1B	13.01	52.47	55.12	53.76	6.11	51.93	35.25	72.84
	ViMoE-VQA	9.55	**62.89**	58.60	**60.69**	12.54	47.12	39.10	88.76
Zero-shot	Vintern-1B	0.12	17.52	19.87	17.55	1.91	25.84	23.93	8.54
	LLaMA 3.2	0.36	23.96	73.71	36.16	3.62	36.11	30.01	62.84
	Gemini 2.0 F.	0.55	27.20	74.10	39.79	4.41	39.60	31.72	74.42
	Gemini 2.5 F.	0.22	24.43	**76.66**	24.75	0.39	37.27	31.22	71.90
	GPT-5	10.84	47.20	55.20	50.89	6.07	47.30	33.34	84.20
Ours	ViBridge-VQA (k=36)	11.75 ±0.21	55.36 ±0.21	56.56 ±0.21	54.56 ±0.19	18.52 ±0.18	52.64 ±0.19	45.53 ±0.33	110.58 ±0.95
	ViBridge-VQA (k=36) + LoRA	**14.67** ±0.30	59.02 ±0.12	59.96 ±0.33	58.17 ±0.19	**22.53** ±0.11	**56.25** ±0.22	**49.32** ±0.37	122.88 ±0.93

one 16 GB NVIDIA T4. Each configuration is trained with three seeds (42, 123, 3407), and we report the mean ± std. The token budget is selected on validation, and the test split is used only for the final report.

Baselines. Table 1 compares with two groups. (1) Multimodal models fine-tuned on AutoViVQA: ViT5_ViT, BARTPhoBEiT [31], Vintern-1B and ViMoE-VQA [21]. (2) Zero-shot models: Vintern-1B, LLaMA 3.2, Gemini 2.0 and 2.5 Flash, and GPT-5. Results of groups (1) and (2) are taken from the AutoViVQA and ViMoE-VQA papers. The fine-tuned Vintern-1B uses the official recipe: six 448 × 448 tiles plus a thumbnail for every AutoViVQA test image, and a rank-16 LoRA on the language model for one epoch. In the analysis, we also use *Global-only*, the bridge of Eq. (1) with $g = 8$ and no local tokens, without and with a decoder LoRA (*Global-only + LoRA*). Our models are *ViBridge-VQA (k=36)*, with $g = 14$, and *ViBridge-VQA (k=36) + LoRA*.

4.3 Main Results

Table 1 compares ViBridge-VQA with the baselines. Without updating any weight of Vintern-1B, ViBridge-VQA raises its F1 from 17.55 (zero-shot) to 54.56. This is comparable to the fine-tuned Vintern-1B (53.76), even on our stricter image-disjoint split (Section 4.1), while the decoder reads 50 instead of 1,792 visual tokens and only 1.35% of the parameters are trained. Only Accuracy is lower (11.75 versus 13.01). On test, the bridge alone with the same 14 global tokens reaches 51.02 F1 (±0.10), so the 36 local tokens add 3.54 F1.

Adding a decoder LoRA raises F1 to 58.17 (+3.61), with 1.57% of the parameters trained. It also gives the best Accuracy (14.67), BLEU (22.53), ROUGE-L (56.25) and METEOR (49.32). ViMoE-VQA, which trains its fusion module and decoder on AutoViVQA, keeps the best Precision and F1. With the same one-epoch LoRA, ViBridge-VQA is 4.74 validation F1 above Global-only (58.21 versus 53.47, Table 2).

Table 2: Comparison of visual-token sources on the validation set (mean ± std over three seeds, four for Global-only; †one run). Params: trained bridge parameters. Best results per block are in bold. All values ×100.

Visual tokens from	Tokens	Params	F1	BLEU	ROUGE-L	METEOR	CIDEr
Bridge only (two epochs)							
Residual	1	4.86M	46.02 ±.36	12.98 ±.15	44.23 ±.38	36.86 ±.31	86.48 ±.77
Global-only	8	7.35M	50.74 ±.17	16.88 ±.23	49.03 ±.12	41.38 ±.21	99.07 ±.47
Tile-Attention	8	4.14M	45.14 ±.92	12.34 ±.32	43.28 ±.95	36.11 ±.73	84.14 ±1.40
Light Q-Former	8	27.6M	47.06 ±.66	13.99 ±.33	45.37 ±.63	37.57 ±.60	88.07 ±1.89
Full Q-Former	16	69.4M	46.84 ±.47	13.76 ±.36	45.10 ±.41	37.43 ±.34	87.74 ±1.24
C-Abstractor†	9	19.87M	49.12	15.46	47.39	39.83	95.01
Patch-pool†	8	0.92M	45.53	12.02	43.59	36.58	86.12
Local tokens only, untrained†	36	0	27.55	2.47	20.82	34.50	67.20
ViBridge-VQA (k=36)	50	12.86M	**54.96** ±.28	**18.73** ±.23	**53.10** ±.27	**46.01** ±.42	**113.29** ±1.01
+ decoder LoRA (one joint epoch, 2.16M parameters)							
Residual	1	4.86M	52.69 ±.14	18.94 ±.36	51.04 ±.13	43.33 ±.33	104.25 ±.93
Global-only	8	7.35M	53.47 ±.17	19.72 ±.09	51.79 ±.15	44.11 ±.23	106.68 ±.69
Tile-Attention	8	4.14M	52.80 ±.06	19.27 ±.16	51.16 ±.06	43.41 ±.05	104.81 ±.14
Light Q-Former	8	27.6M	53.21 ±.14	19.92 ±.31	51.54 ±.13	43.77 ±.19	106.26 ±.73
Full Q-Former	16	69.4M	53.29 ±.29	19.57 ±.25	51.62 ±.30	43.96 ±.21	106.50 ±1.04
Local tokens only	36	0	xx.xx ±.xx	xx.xx ±.xx	xx.xx ±.xx	xx.xx ±.xx	xx.xx ±.xx
ViBridge-VQA (k=36)	50	12.86M	**58.21** ±.16	**22.27** ±.17	**56.42** ±.14	**49.55** ±.06	**124.88** ±.75

Results are stable across seeds: on test, the std is at most 0.19 for F1 and 0.95 for CIDEr, and test F1 is within 0.4 of validation F1.

4.4 Source of the Visual Tokens

Table 2 compares three sources of visual tokens. Two bridges read the `[CLS]` vector: *Residual* maps it to one token with a linear layer plus a residual MLP, and *Global-only* is the bridge of Eq. (1) with $g = 8$. Five bridges learn to compress the 577 encoder tokens. *Tile-Attention* projects them linearly, refines them with one self-attention layer and reads them with eight learned queries. The light Q-Former [13] refines seven learned queries with two transformer layers that attend to the projected tokens, and adds one token from their mean. The full Q-Former uses 15 queries and four layers that also attend to the question. C-Abstractor [4] applies two residual convolution blocks, pools the token grid to 3×3 and applies two more blocks. It uses 9 tokens, far fewer than the 144 or more of Honeybee. *Patch-pool* projects each token linearly and averages consecutive tokens into eight. All bridges are randomly initialised, as when a new connector is learned for a low-resource task, and trained with the same recipe on the same questions. The third source is our local tokens from the pre-trained projector, used alone ($g = 0$) or with global tokens (ViBridge-VQA).

Reusing the projector works best. Without LoRA, the learned compressions stay 1.6–5.6 F1 below Global-only, despite up to 9.4× more parameters. Patch-pool, a minimal baseline that averages bands of consecutive encoder tokens after a newly learned projection, reaches 45.53 F1. It differs from our local tokens in the pooling (eight 1D bands instead of a 2D grid) as well as in the projection. In contrast, adding 36 or 144 local tokens to Global-only ($g = 8$) raises F1 by 3.58

Table 3: Effect of the token budget on the validation set (mean ± std over three seeds, four for $g = 8$, $k = 0$). Time: minutes to generate the validation answers on one T4. The selected budget is in bold. All values ×100.

g	k	Tokens	Time	F1	BLEU	ROUGE-L	METEOR	CIDEr
8	0	8	33.9	50.74 ±.17	16.88 ±.23	49.03 ±.12	41.38 ±.21	99.07 ±.47
14	0	14	37.2	51.56 ±.08	17.76 ±.01	49.86 ±.05	42.14 ±.17	101.18 ±.52
8	1	9	34.2	51.03 ±.12	16.54 ±.27	49.21 ±.13	41.37 ±.11	99.27 ±.58
8	9	17	35.5	52.82 ±.11	17.58 ±.03	50.96 ±.05	43.56 ±.20	106.11 ±.55
8	36	44	36.0	54.32 ±.41	18.22 ±.39	52.48 ±.43	45.31 ±.42	111.88 ±1.43
8	144	152	41.7	56.15 ±.08	19.82 ±.00	54.21 ±.06	47.51 ±.05	119.05 ±.25
14	1	15	35.3	51.67 ±.25	17.30 ±.36	49.93 ±.24	42.36 ±.19	101.66 ±1.11
14	9	23	34.7	52.98 ±.08	17.83 ±.23	51.17 ±.10	43.71 ±.13	106.02 ±.80
14	**36**	**50**	**35.8**	**54.96** ±.28	**18.73** ±.23	**53.10** ±.27	**46.01** ±.42	**113.29** ±1.01
14	144	158	41.1	56.61 ±.04	20.18 ±.13	54.70 ±.05	48.00 ±.05	120.71 ±.12

and 5.41 (Table 3). The projector also wins at a similar number of tokens: 17 tokens ($g = 8$, $k = 9$) give 52.82 F1, against 46.84 for the 16-token full Q-Former, and 9 tokens ($g = 8$, $k = 1$) give 51.03, against 49.12 for the 9-token C-Abstractor. We also tried to distil the projector into Global-only. A cosine loss between the mean tokens of the bridge and of the projector gives 50.35 F1, and a KL loss between the answer distributions obtained with each gives 50.46. Neither reaches the 50.74 obtained without distillation.

Local tokens and LoRA are complementary. With a one-epoch decoder LoRA, all bridges without local tokens reach 52.69–53.47 F1, below ViBridge-VQA with a frozen decoder (54.96). Trained for three epochs, Global-only + LoRA reaches 54.78, on par with it. The same one-epoch LoRA adds 3.25 F1 to ViBridge-VQA, more than the 2.73 it adds to Global-only. Without global tokens ($g = 0$), the local tokens give 27.55 F1 with nothing trained, as the frozen model answers in long sentences (24.8 words on average), and xx.xx with the same LoRA, against 58.21 with the 14 global tokens.

4.5 Visual Token Budget

Local tokens matter more than global tokens. Table 3 and Fig. 2a vary $g \in \{8, 14\}$ and $k \in \{1, 9, 36, 144\}$. At $g = 14$, going from 1 to 144 local tokens adds 4.94 F1 and 19.05 CIDEr, whereas going from 8 to 14 global tokens adds only 0.2–0.8 F1 at a fixed k. The gain per token falls from 0.07 F1 (23 to 50 tokens) to 0.015 F1 (50 to 158 tokens).

Selected budget. F1 grows with every added token, so all configurations with local tokens lie on the accuracy–token Pareto front. Its knee (Section 3.5) is $g = 14$, $k = 36$ for every metric, and also when generation time is the cost, although times differ by only a few minutes below $k = 144$. TOPSIS with equal weights

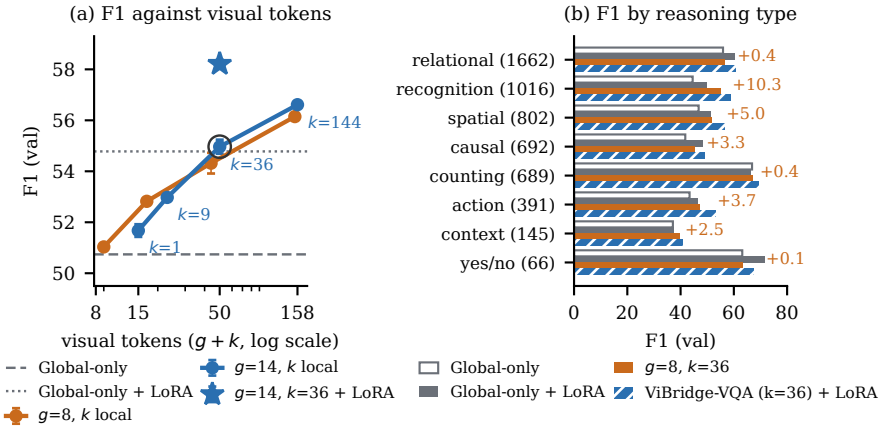


Fig. 2: (a) Validation F1 against the number of visual tokens (circle: selected budget). (b) Validation F1 by reasoning type (number of questions); numbers: gain of 36 local tokens at $g = 8$, the g of Global-only.

Table 4: Global-only bridge with g tokens and no local tokens, on the validation set (mean ± std over three seeds, four for $g = 8$; †two seeds). The selected g is in bold. All values ×100.

g	F1	CIDEr	g	F1	CIDEr
4	48.98 ±.52	93.56 ±1.42	**14**	**51.56** ±.08	**101.18** ±.52
6	49.58 ±.47	95.51 ±1.59	16	51.20 ±.39	100.29 ±.95
8	50.74 ±.17	99.07 ±.47	18	51.26 ±.73	100.46 ±2.02
10	51.01 ±.10	99.20 ±.25	20†	51.60 ±.08	101.29 ±.22
12	51.10 ±.13	99.26 ±.54			

and the post-hoc knee on test also select this budget. $k = 144$ is 1.65 F1 higher, but $k = 36$ obtains 72% of its gain over Global-only with 32% of its tokens (67% against the 14-token Global-only bridge of Table 4, to which the 36 local tokens add 3.40 F1).

Number of global tokens. Table 4 varies the number of global tokens of Global-only from $g = 4$ to $g = 20$ (three seeds, two for $g = 20$). Validation F1 rises from 48.98 ($g = 4$) to 50.74 ($g = 8$) and 51.56 ($g = 14$), then stays between 51.20 and 51.60 for $g = 16$–20. CIDEr follows the same trend, so we combine the local tokens with $g = 14$ and, as the Global-only reference, $g = 8$.

Computational cost. The decoder reads 50 visual tokens, against 1,792 for the official tiling (six tiles plus a thumbnail for every test image). The encoder processes one 336×336 view (191 GMACs) instead of seven 448×448 views (362 GMACs each), 13 times less computation. Within our budgets, $k = 144$ instead of $k = 36$ makes generation about 15% slower.



Fig. 3: Qualitative examples from the validation set. GT: one of the five reference answers; Ours: ViBridge-VQA (k=36); Ours+LoRA: ViBridge-VQA (k=36) + LoRA.

4.6 Analysis

Reasoning types. Fig. 2b breaks down validation F1 by reasoning type. Local tokens help most when the answer is visible in the image: at $g = 8$, 36 local tokens add 10.3 F1 on recognition, 5.0 on spatial and 3.7 on action questions. On these types, the frozen-decoder model even beats Global-only + LoRA. Decoder adaptation helps more on causal and relational questions, which have the longest answers (5.0 and 5.6 words), and on the 66 yes/no questions. ViBridge-VQA (k=36) + LoRA is the best of the four models on seven of the eight types, all but yes/no.

Error analysis. On validation, our answers are as long as the references (4.3 words on average), and most of them (77–81%) partially overlap the closest reference. Compared with Global-only, ViBridge-VQA raises exact matches (F1 = 1) from 9.7% to 13.3%. It also lowers answers with no word in common with any reference from 9.6% to 8.0% (6.8% with LoRA).

Qualitative examples. Fig. 3 shows four validation examples. ViBridge-VQA already finds the relevant content, such as the driver's phone or the helmets, but its answers are vague. The decoder LoRA makes them precise: the driver is using *the phone*, and the helmets are worn *to protect their heads*.

Table 5: Cross-dataset results (mean ± std over three samples of about 1,000 test questions, one training seed). ViBridge-VQA (k=144): the most accurate budget of Table 3, without LoRA. Best results per dataset are in bold. All values ×100.

Model	F1	BLEU	ROUGE-L	METEOR	CIDEr
ViTextVQA					
Vintern-1B, zero-shot	32.01 ±.31	4.94 ±.18	20.89 ±.25	26.50 ±.54	**195.62** ±2.35
Global-only + LoRA	3.96 ±.09	0.15 ±.01	3.74 ±.12	2.61 ±.16	15.64 ±.92
ViBridge-VQA (k=36)	15.14 ±.47	1.01 ±.10	14.27 ±.43	10.49 ±.36	66.39 ±2.51
ViBridge-VQA (k=36) + LoRA	15.39 ±.39	1.03 ±.15	14.41 ±.33	11.17 ±.26	70.07 ±1.33
ViBridge-VQA (k=144)	**33.26** ±.67	**6.24** ±.48	**31.65** ±.73	**27.33** ±.93	190.30 ±6.71
ViVQA-X					
Vintern-1B, zero-shot	14.52 ±.47	0.06 ±.01	8.98 ±.32	11.96 ±.41	50.13 ±2.83
Global-only + LoRA	15.89 ±.65	0.18 ±.02	11.94 ±.49	9.08 ±.57	42.12 ±2.92
ViBridge-VQA (k=36)	24.91 ±.14	0.34 ±.12	19.38 ±.25	16.00 ±.21	79.51 ±1.15
ViBridge-VQA (k=36) + LoRA	27.64 ±.11	0.33 ±.05	21.98 ±.13	**19.24** ±.14	**95.29** ±1.51
ViBridge-VQA (k=144)	**27.89** ±.13	**0.39** ±.13	**22.02** ±.26	18.55 ±.11	92.22 ±2.31
ViVQA					
Vintern-1B, zero-shot	23.90 ±.24	0.27 ±.02	15.78 ±.25	17.84 ±.19	78.63 ±2.07
Global-only + LoRA	31.48 ±1.15	0.25 ±.09	31.48 ±1.16	21.92 ±.82	84.94 ±2.87
ViBridge-VQA (k=36)	42.68 ±1.52	0.37 ±.00	42.65 ±1.55	34.62 ±1.37	171.21 ±6.06
ViBridge-VQA (k=36) + LoRA	44.76 ±1.21	**0.63** ±.02	44.74 ±1.22	36.98 ±1.28	183.60 ±7.17
ViBridge-VQA (k=144)	**44.95** ±.90	0.59 ±.17	**44.95** ±.90	**37.28** ±.61	**191.28** ±2.17

4.7 Generalisation to Other Datasets

Table 5 tests the models on three Vietnamese VQA datasets not used for training. ViTextVQA [24] focuses on scene text, while ViVQA-X [9] and ViVQA [30] are general-domain datasets on COCO images. About 10% of their sampled questions concern images of our training split, with different questions. On the general-domain datasets, ViBridge-VQA (k=36) outperforms Global-only + LoRA and the zero-shot Vintern-1B even without LoRA (F1 24.91 versus 14.52 on ViVQA-X, 42.68 versus 23.90 on ViVQA), and LoRA raises its F1 to 27.64 and 44.76. Scene text needs more local tokens. With 36 of them, the model stays far below Vintern-1B (15.39 versus 32.01 F1), whereas ViBridge-VQA (k=144) is on par with it (33.26) while reading one view instead of up to seven, suggesting that the gap comes mainly from the token budget. ViBridge-VQA (k=144) is also the best or on par on the general-domain datasets. Averaged over the three datasets, F1 is 35.4 for ViBridge-VQA (k=144) and 29.3 for ViBridge-VQA (k=36) + LoRA, against 23.5 for Vintern-1B.

5 Conclusion

In this work, we introduced **ViBridge-VQA**, which improves the Vietnamese VLM Vintern-1B for generative VQA without updating any of its pre-trained weights. Our results support three findings for a VLM whose projector is already aligned with its decoder. First, reusing this projector with a few pooled local tokens is more effective than learning a new connector from limited data. Second, the number of tokens can be chosen with an explicit accuracy–cost criterion. Third, the local tokens complement decoder adaptation rather than replace it.

Our study also has limitations. We train on a single dataset, AutoViVQA, and the task-specific ViMoE-VQA still reaches a higher F1. The comparison with published results is also limited, as they were obtained on the original AutoViVQA split. Without decoder adaptation, the exact-match accuracy of ViBridge-VQA stays below that of the fine-tuned Vintern-1B. With 50 visual tokens, it also reads scene text much worse than Vintern-1B, a gap that closes with all 144 local tokens. Finally, accuracy has not saturated at the 144 local tokens available at our resolution.

Overall, an existing Vietnamese VLM can be improved substantially by choosing which visual tokens its frozen decoder receives, and how many. The code and the image-disjoint split are available at `https://github.com/nguyennn263/modular-vlm-finetune`.

References

1. Alayrac, J.B., Donahue, J., Luc, P., Miech, A., Barr, I., Hasson, Y., Lenc, K., Mensch, A., Millican, K., Reynolds, M., et al.: Flamingo: a visual language model for few-shot learning. In: Advances in Neural Information Processing Systems (NeurIPS). pp. 23716–23736 (2022)
2. Banerjee, S., Lavie, A.: METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In: Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization. pp. 65–72 (2005)
3. Cai, M., Yang, J., Gao, J., Lee, Y.J.: Matryoshka multimodal models. In: Proceedings of the International Conference on Learning Representations (ICLR) (2025)
4. Cha, J., Kang, W., Mun, J., Roh, B.: Honeybee: Locality-enhanced projector for multimodal LLM. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 13817–13827 (2024)
5. Chen, L., Zhao, H., Liu, T., Bai, S., Lin, J., Zhou, C., Chang, B.: An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In: Proceedings of the European Conference on Computer Vision (ECCV) (2024)
6. Chen, Z., Wang, W., Tian, H., Ye, S., Gao, Z., Cui, E., Tong, W., Hu, K., Luo, J., Ma, Z., et al.: How far are we to GPT-4V? closing the gap to commercial multimodal models with open-source suites. Science China Information Sciences **67**(12), 220101 (2024)
7. Chen, Z., Wu, J., Wang, W., Su, W., Chen, G., Xing, S., Zhong, M., Zhang, Q., Zhu, X., Lu, L., Li, B., Luo, P., Lu, T., Qiao, Y., Dai, J.: InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 24185–24198 (2024)
8. Doan, K.T., Huynh, B.G., Hoang, D.T., Pham, T.D., Pham, N.H., Nguyen, Q.T.M., Vo, B.Q., Hoang, S.N.: Vintern-1B: An efficient multimodal large language model for Vietnamese (2024), arXiv preprint arXiv:2408.12480
9. Duong, T.B., Tran, H.M., Le-Nguyen, B.N., Duong, D.T.: An automated pipeline for constructing a Vietnamese VQA-NLE dataset. In: Proceedings of the International Conference on Intelligent Systems and Networks (ICISN). pp. 164–173. Lecture Notes in Networks and Systems, Springer (2026)

10. Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M., Gelly, S.: Parameter-efficient transfer learning for NLP. In: Proceedings of the International Conference on Machine Learning (ICML). pp. 2790–2799 (2019)
11. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: LoRA: Low-rank adaptation of large language models. In: Proceedings of the International Conference on Learning Representations (ICLR) (2022)
12. Hwang, C.L., Yoon, K.: Multiple Attribute Decision Making: Methods and Applications. Springer (1981)
13. Li, J., Li, D., Savarese, S., Hoi, S.: BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In: Proceedings of the International Conference on Machine Learning (ICML). pp. 19730–19742 (2023)
14. Li, K.Y., Goyal, S., Semedo, J.D., Kolter, J.Z.: Inference optimal VLMs need only one visual token but larger models (2024), arXiv preprint arXiv:2411.03312
15. Li, W., Yuan, Y., Liu, J., Tang, D., Wang, S., Qin, J., Zhu, J., Zhang, L.: TokenPacker: Efficient visual projector for multimodal LLM (2024), arXiv preprint arXiv:2407.02392
16. Li, X.L., Liang, P.: Prefix-tuning: Optimizing continuous prompts for generation. In: Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL). pp. 4582–4597 (2021)
17. Lin, C.Y.: ROUGE: A package for automatic evaluation of summaries. In: Proceedings of the ACL Workshop on Text Summarization Branches Out. pp. 74–81 (2004)
18. Lin, T.Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C.L., Dollár, P.: Microsoft COCO: Common objects in context. In: Proceedings of the European Conference on Computer Vision (ECCV). pp. 740–755 (2014)
19. Liu, H., Li, C., Li, Y., Lee, Y.J.: Improved baselines with visual instruction tuning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 26296–26306 (2024)
20. Liu, H., Li, C., Wu, Q., Lee, Y.J.: Visual instruction tuning. In: Advances in Neural Information Processing Systems (NeurIPS). pp. 34892–34916 (2023)
21. Nguyen, A.T., Nguyen, T.Q., Dang, D.L., Tran, D.T., Phan, B.D., Nguyen, Q.T., Le, T.: ViMoE-VQA: Reasoning-aware sparse experts for generative Vietnamese visual question answering. Procedia Computer Science (2026), to appear
22. Nguyen, A.T., Phan, B.D., Nguyen, T.Q., Tran, D.T., Dang, D.L., Nguyen, Q.T., Le, T.: AutoViVQA: A large-scale, automatically constructed dataset for Vietnamese visual question answering. In: Proceedings of the Asian Conference on Intelligent Information and Database Systems (ACIIDS). pp. 492–506. Communications in Computer and Information Science, Springer (2026)
23. Nguyen, N.S., Nguyen, V.S., Le, T.: Advancing Vietnamese visual question answering with transformer and convolutional integration. Computers and Electrical Engineering **119**, 109474 (2024)
24. Nguyen, Q.V., Tran, D.Q., Pham, H.Q., Nguyen, T.K.B., Nguyen, N.H., Nguyen, K.V., Nguyen, N.L.T.: ViTextVQA: A large-scale visual question answering dataset for evaluating Vietnamese text comprehension in images (2025), arXiv preprint arXiv:2404.10652
25. Papineni, K., Roukos, S., Ward, T., Zhu, W.J.: BLEU: a method for automatic evaluation of machine translation. In: Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL). pp. 311–318 (2002)

26. Pham, H.Q., Nguyen, T.K.B., Nguyen, Q.V., Tran, D.Q., Nguyen, N.H., Nguyen, K.V., Nguyen, N.L.T.: ViOCRVQA: Novel benchmark dataset and vision reader for visual question answering by understanding Vietnamese text in images (2024), arXiv preprint arXiv:2404.18397
27. Qwen Team: Qwen2.5 technical report (2024), arXiv preprint arXiv:2412.15115
28. Satopää, V., Albrecht, J., Irwin, D., Raghavan, B.: Finding a "kneedle" in a haystack: Detecting knee points in system behavior. In: Proceedings of the International Conference on Distributed Computing Systems Workshops (ICDCSW). pp. 166–171 (2011)
29. Shang, Y., Cai, M., Xu, B., Lee, Y.J., Yan, Y.: LLaVA-PruMerge: Adaptive token reduction for efficient large multimodal models (2024), arXiv preprint arXiv:2403.15388
30. Tran, K.Q., Nguyen, A.T., Le, A.T.H., Nguyen, K.V.: ViVQA: Vietnamese visual question answering. In: Proceedings of the Pacific Asia Conference on Language, Information and Computation (PACLIC). pp. 683–691 (2021)
31. Tran, K.V., Nguyen, K.V., Nguyen, N.L.T.: BARTPhoBEiT: Pre-trained sequence-to-sequence and image transformers models for Vietnamese visual question answering. In: Proceedings of the International Conference on Multimedia Analysis and Pattern Recognition (MAPR). pp. 1–6 (2023)
32. Tran, K.V., Phan, H.P., Nguyen, K.V., Nguyen, N.L.T.: ViCLEVR: A visual reasoning dataset and hybrid multimodal fusion model for visual question answering in Vietnamese (2023), arXiv preprint arXiv:2310.18046
33. Tsimpoukelli, M., Menick, J., Cabi, S., Eslami, S.M.A., Vinyals, O., Hill, F.: Multimodal few-shot learning with frozen language models. In: Advances in Neural Information Processing Systems (NeurIPS). pp. 200–212 (2021)
34. Vedantam, R., Zitnick, C.L., Parikh, D.: CIDEr: Consensus-based image description evaluation. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 4566–4575 (2015)
35. Yang, S., Chen, Y., Tian, Z., Wang, C., Li, J., Yu, B., Jia, J.: VisionZip: Longer is better but not necessary in vision language models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (2025)
36. Yao, L., Li, L., Ren, S., Wang, L., Liu, Y., Sun, X., Hou, L.: DeCo: Decoupling token compression from semantic abstraction in multimodal large language models (2024), arXiv preprint arXiv:2405.20985